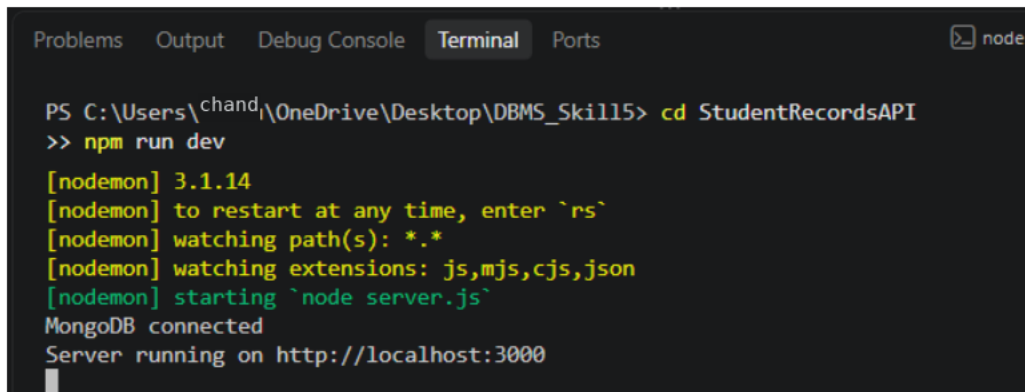


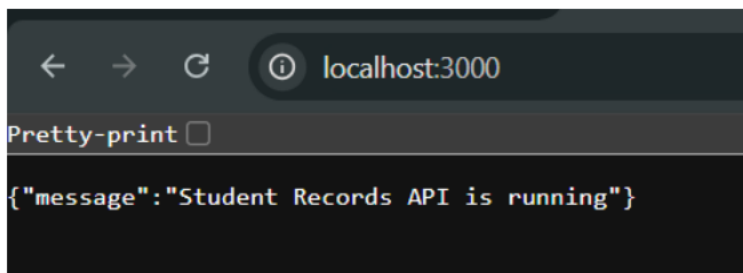
DBMS Practical – 7

SERVER running:



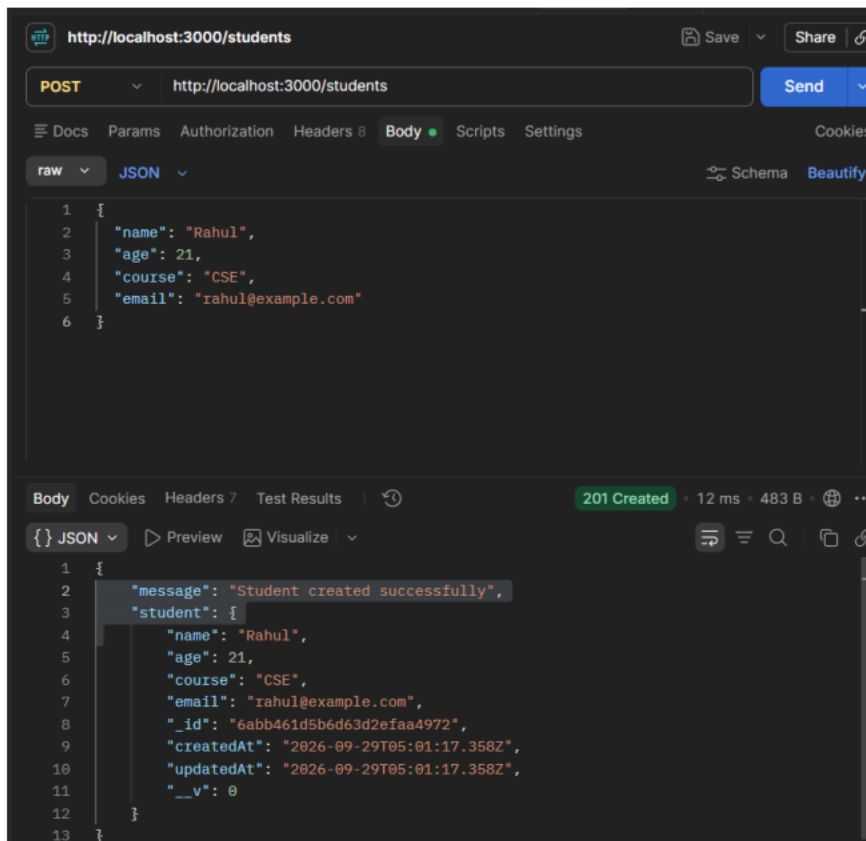
```
PS C:\Users\chand\OneDrive\Desktop\DBMS_Skill15> cd StudentRecordsAPI
>> npm run dev
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
MongoDB connected
Server running on http://localhost:3000
```

API Running:



```
localhost:3000
Pretty-print ☐
{"message":"Student Records API is running"}
```

POST:



```
http://localhost:3000/students
POST http://localhost:3000/students
Body
raw JSON
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
201 Created - 12 ms - 483 B
Body
1 {
2   "message": "Student created successfully",
3   "student": {
4     "name": "Rahul",
5     "age": 21,
6     "course": "CSE",
7     "email": "rahul@example.com",
8     "_id": "6abb461d5b6d63d2efaa4972",
9     "createdAt": "2026-09-29T05:01:17.358Z",
10    "updatedAt": "2026-09-29T05:01:17.358Z",
11    "__v": 0
12  }
13 }
```

GET:

http://localhost:3000/students

GET http://localhost:3000/students

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

```
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
```

Body Cookies Headers 7 Test Results

{ } JSON Preview Visualize

```
1 {
2   "_id": "6abb42ae5b6d63d2efaa496f",
3   "name": "Anita",
4   "age": 20,
5   "course": "CSE",
6   "email": "anita@example.com",
7   "createdAt": "2026-09-29T04:46:38.600Z",
8   "updatedAt": "2026-09-29T04:46:38.600Z",
9   "__v": 0
10 },
11 {
12   "_id": "6abb461d5b6d63d2efaa4972",
13   "name": "Rahul",
14   "age": 21,
15   "course": "CSE",
16   "email": "rahul@example.com",
17   "createdAt": "2026-09-29T05:01:17.358Z",
18   "updatedAt": "2026-09-29T05:01:17.358Z",
19   "v": 0
20 }
```

http://localhost:3000/students/6abb461d5b6d63d2efaa4972

GET http://localhost:3000/students/6abb461d5b6d63d2efaa4972

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JS

```
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
```

Body Cookies Headers 7 Test Results

{ } JSON Preview Visualize

```
1 {
2   "_id": "6abb461d5b6d63d2efaa4972",
3   "name": "Rahul",
4   "age": 21,
5   "course": "CSE",
6   "email": "rahul@example.com",
7   "createdAt": "2026-09-29T05:01:17.358Z",
8   "updatedAt": "2026-09-29T05:01:17.358Z",
9   "__v": 0
10 }
```

PATCH:

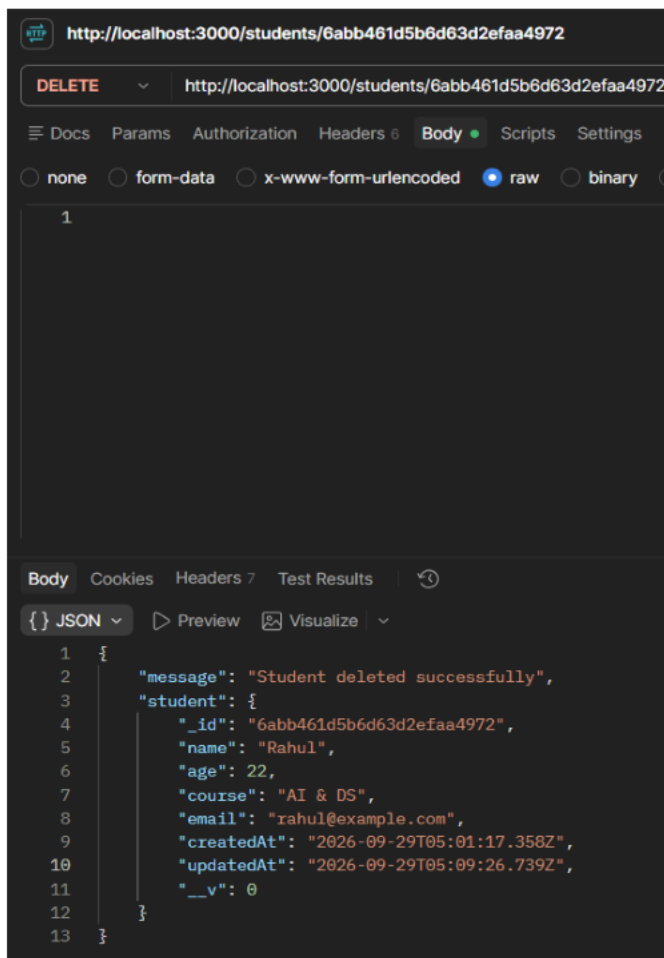
The screenshot shows an HTTP client interface with the following details:

- URL:** `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`
- Method:** `PATCH`
- Body Type:** `raw` (selected)
- Request Body:**

```
1 {  
2   "age": 22,  
3   "course": "AI & DS"  
4 }
```
- Response Body:** `JSON` (selected)

```
1 {  
2   "message": "Student updated successfully",  
3   "student": {  
4     "_id": "6abb461d5b6d63d2efaa4972",  
5     "name": "Rahul",  
6     "age": 22,  
7     "course": "AI & DS",  
8     "email": "rahul@example.com",  
9     "createdAt": "2026-09-29T05:01:17.358Z",  
10    "updatedAt": "2026-09-29T05:07:57.556Z",  
11    "__v": 0  
12  }  
13 }
```

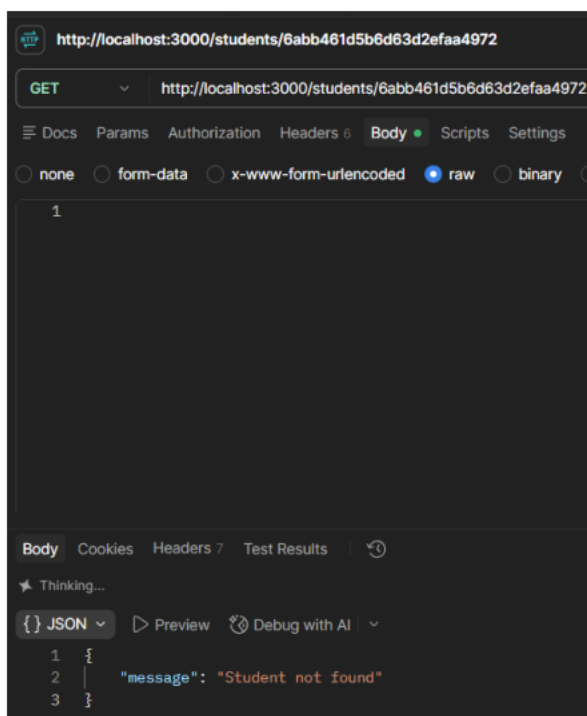
DELETE:



The image shows a Postman interface for a DELETE request. The URL bar at the top displays `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`. Below the URL bar, the method is set to **DELETE**. The 'Body' tab is selected, and the 'raw' radio button is chosen. The body content is a single line with the number `1`. At the bottom, the 'JSON' tab is selected, showing a response body with the following structure:

```
1 {
2   "message": "Student deleted successfully",
3   "student": {
4     "_id": "6abb461d5b6d63d2efaa4972",
5     "name": "Rahul",
6     "age": 22,
7     "course": "AI & DS",
8     "email": "rahul@example.com",
9     "createdAt": "2026-09-29T05:01:17.358Z",
10    "updatedAt": "2026-09-29T05:09:26.739Z",
11    "__v": 0
12  }
13 }
```

GET (VERIFY):



The image shows a Postman interface for a GET request. The URL bar at the top displays `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`. Below the URL bar, the method is set to **GET**. The 'Body' tab is selected, and the 'raw' radio button is chosen. The body content is a single line with the number `1`. At the bottom, the 'JSON' tab is selected, showing a response body with the following structure:

```
1 {
2   "message": "Student not found"
3 }
```

Duplicate Email Validation:

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/students`
- Method:** `POST`
- Body (raw):**

```
1 {
2   "name": "Another Anita",
3   "age": 22,
4   "course": "ECE",
5   "email": "anita@example.com"
6 }
```
- Response:** `400 Bad Request` (6 ms, 416 B)
- Response Body (JSON):**

```
1 {
2   "message": "Failed to create student",
3   "error": "E11000 duplicate key error collection: student_records.students index: email_1 dup
4             key: { email: \"anita@example.com\" }"
```

MongoDB Compass:

The screenshot shows the MongoDB Compass interface with the following details:

- Database:** `student_records`
- Collection:** `students`
- Documents:** 1 document is displayed.
- Document Content:**

```
_id: ObjectId('6abb42ae5b6d63d2efaa496f')
name: "Anita"
age: 20
course: "CSE"
email: "anita@example.com"
createdAt: ISODate('2026-09-29T04:46:38.600+00:00')
updatedAt: ISODate('2026-09-29T04:46:38.600+00:00')
```